

RAG-Powered AI Tutoring System: A Dual-Interface Retrieval-Augmented Generation
Approach for Course-Specific Learning

Sachet Kanchugar, Akash Deep Singh, and Sanjeet Srinivasa

Northeastern University

Submitted in partial fulfillment of the requirements for Foundations of Artificial
Intelligence

Abstract

Large language models (LLMs) have demonstrated remarkable capabilities across a broad range of natural language tasks, yet their tendency to hallucinate factual content poses a serious challenge in educational settings where accuracy and source attribution are paramount. This paper describes the design, implementation, and evaluation of a Retrieval-Augmented Generation (RAG) powered AI tutoring system built specifically for course-level learning environments. The system features a dual-interface architecture: an instructor-facing dashboard through which teachers upload course documents, and a student-facing conversational chatbot that generates answers grounded exclusively in those materials. Documents are parsed across PDF, plain text, and Markdown formats, chunked, embedded using OpenAI’s `text-embedding-3-small` model, and stored in a persistent FAISS vector index. At query time, the top three retrieved chunks are passed to GPT-4o-mini with a grounding-enforcing system prompt and `temperature=0` for deterministic output. When a question falls outside the course corpus, a consent-gated LLM fallback mechanism explicitly notifies the student and requires opt-in before returning an ungrounded response. The system additionally integrates JWT-based role authentication with a SQLite user database and a React front-end. We present a qualitative evaluation across factual recall, conceptual synthesis, and edge-case queries, and discuss the design’s strengths and directions for future improvement.

Keywords: retrieval-augmented generation, educational technology, large language models, hallucination, vector retrieval, chatbot

RAG-Powered AI Tutoring System: A Dual-Interface Retrieval-Augmented Generation Approach for Course-Specific Learning

Introduction

The rapid proliferation of large language models has prompted growing interest in their application to educational technology. Systems such as GPT-4 are capable of answering complex questions and generating pedagogically coherent text across virtually any subject domain. Despite these capabilities, deploying a general-purpose LLM directly as a course tutor exposes a fundamental limitation: such models generate answers from their pre-trained parameters alone, with no access to the specific documents or readings that constitute a given course. The result is a system that may sound authoritative whilst contradicting or ignoring the material students are expected to learn. This problem, commonly referred to as hallucination, is a central concern in educational contexts, since a student who receives a confident but incorrect explanation may internalise misconceptions that are difficult to correct later.

Retrieval-Augmented Generation (RAG), introduced by Lewis et al. (2020), provides a principled framework for addressing this problem. Rather than relying solely on parametric knowledge, RAG augments generation with a dense vector index over an external document corpus. At inference time, the system retrieves the most semantically relevant passages in response to a user query and conditions generation on those passages. Lewis et al. (2020) demonstrated that this approach substantially reduces hallucination and improves factual accuracy on knowledge-intensive tasks, and noted that surfacing retrieved passages alongside generated answers provides a degree of interpretability absent from purely parametric models.

Our project adapts this framework to the educational domain. We built a fully functional RAG-powered tutoring system with two distinct interfaces: an instructor dashboard through which teachers upload course documents, and a student-facing chatbot that generates answers grounded exclusively in those materials. The system stores documents in a FAISS vector index (Johnson et al., 2017), supports PDF, plain text, and Markdown files, and introduces a consent-gated LLM fallback for questions

outside the course corpus. Section 2 describes the architecture and implementation. Section 3 presents a qualitative evaluation. Section 4 summarises contributions and future directions.

Methods

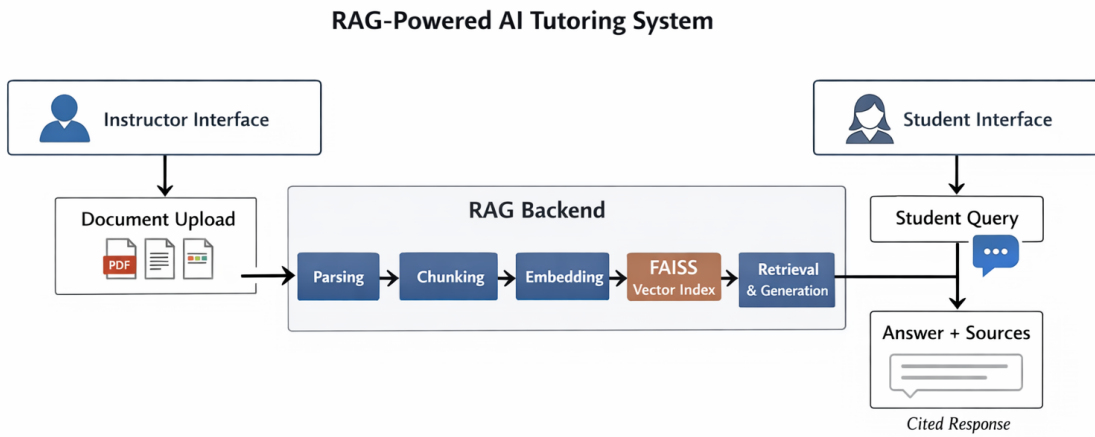


Figure 1. System architecture of the RAG-powered tutoring system. The instructor interface (left) drives the ingestion pipeline: documents are parsed, chunked, embedded, and stored in a FAISS vector index. The student interface (right) submits natural language queries that trigger retrieval and generation, returning a cited response grounded in the indexed course materials.

Our system is organised around a RAG pipeline extended in several directions for educational deployment. Each component was selected and configured with the specific constraints of a course-level tutoring context in mind: the corpus is small and domain-specific, queries are student questions rather than keyword searches, and outputs must be source-attributed rather than merely fluent. The backend is implemented in Python using FastAPI (Ramírez, 2023) and comprises eight modules:

five core application modules (`main.py` for API routing, `rag.py` for retrieval and generation, `document_loader.py` for parsing and chunking, `vector_store.py` for FAISS management, and `auth.py` for access control) and three supporting modules (`database.py` for SQLAlchemy models, `jwt_utils.py` for token creation and verification, and `password_utils.py` for password hashing). The frontend is a React (React Core Team, 2024) application styled with TailwindCSS.

Document Processing Pipeline

Three file types are supported: PDFs are processed using LangChain’s `PyPDFLoader` (LangChain Development Team, 2023), which extracts text page-by-page and preserves the actual page number as metadata, enabling precise source citations; plain text and Markdown files are handled with `TextLoader`, which does not produce page-level splits, so these files receive a default page value of 1 and citations reference the document name rather than a specific page.

Supporting Markdown as a first-class format was a deliberate design decision: many technical course materials, including problem sets and assignment specifications, are authored in Markdown, and treating them natively avoids the artefacts introduced by converting Markdown to PDF before ingestion.

Parsed text is split into overlapping chunks of 500 characters with 100 characters of overlap. This size was chosen to match the typical granularity of a student question: short enough for a single chunk to correspond to a coherent sub-topic, long enough to preserve sentence-level context. The 20% overlap ensures that concepts straddling chunk boundaries appear in full in at least one chunk. Document filename and page number are attached to each chunk as metadata at ingestion time and preserved through embedding and indexing, so every retrieved chunk carries a traceable provenance record.

Each chunk is embedded using OpenAI’s `text-embedding-3-small` (OpenAI, 2024), chosen for its strong semantic similarity on short domain-specific passages without the latency cost of larger models. Vectors are stored in a persistent FAISS index (Johnson et al., 2017) managed as a lazy-loading singleton: loaded from disk into

memory on the first request and reused across all subsequent requests. New documents are added incrementally rather than triggering a full index rebuild, and if no documents have been uploaded, retrieval returns an empty list gracefully rather than raising an error.

Retrieval and Generation

At query time, the student’s question is embedded with the same `text-embedding-3-small` model, ensuring document chunks and queries occupy the same vector space. The top three chunks are retrieved by similarity search over the FAISS index. Three was chosen to balance context richness against prompt length: fewer chunks risk omitting relevant material; more risk diluting the prompt with loosely related passages.

Retrieved chunks are assembled into a prompt passed to GPT-4o-mini (OpenAI, 2024) with a carefully engineered system instruction: the model is assigned the role of an AI tutor, instructed to answer exclusively from the provided context, and required to return the exact phrase “I cannot find this information in the course materials” when no relevant context is present. GPT-4o-mini was selected over full GPT-4o because the grounding constraint eliminates the need for broad parametric knowledge; in this setting GPT-4o-mini provides adequate reasoning at substantially lower latency and cost. The model is invoked with `temperature=0` to prioritise determinism over stylistic variation, as two students asking the same question should receive the same grounded answer.

When the refusal phrase is detected, the system returns a `llm_fallback_available` flag and a plain-language disclaimer. The student may explicitly opt in to an ungrounded AI-generated answer by resubmitting with `llm_fallback: true`. This consent-gated design ensures students are never silently served answers outside the course corpus. Each response includes the document name, page number, and a 300-character snippet for all three retrieved chunks, surfacing the full retrieval context so students can inspect every passage the answer was drawn from, directly reflecting the interpretability insight of Lewis et al. (2020).

Authentication and Role Separation

The system uses JWT-based authentication with `POST /auth/signup` and `POST /auth/signin` endpoints. User accounts are persisted in a SQLite database via SQLAlchemy; the `users` table stores each user’s email (primary key), display name, hashed password, role enum (`student` or `teacher`), and creation timestamp. On sign-in, the server verifies the password hash, confirms the role matches the account’s registered role, and issues an HS256-signed JWT with a 24-hour expiry.

Three further endpoints are role-gated via FastAPI dependency injection: `POST /upload` is restricted to teachers, `GET /documents` and `POST /chat` are available to any authenticated user. This departs from the original RAG framework of Lewis et al. (2020), which makes no distinction between those who contribute knowledge and those who query it. By grounding access control in database-backed accounts, the system supports multiple instructors simultaneously, each with individually auditable upload histories, ensuring the retrieval corpus remains curated and pedagogically appropriate.

Frontend Design

The React frontend routes users to role selection, sign-in, or sign-up screens when unauthenticated. The JWT is stored in the browser and attached to every API request. The teacher-facing `UploadPage` presents a file picker restricted to `.pdf`, `.txt`, and `.md`, mirroring the server-side constraint. After a successful upload it re-fetches the document list and displays each entry with filename, type, and timestamp, providing instructors with a clear audit of the current corpus.

The student-facing `ChatPage` renders a scrolling message thread with user messages as right-aligned bubbles and assistant responses as left-aligned cards. Source citations appear beneath each response with document name, page number, and a verbatim 300-character snippet. When the fallback flag is set, a native `window.confirm()` dialog asks the student to explicitly consent before an ungrounded answer is served. The input submits on Enter, with Shift+Enter for newlines, and both components surface server error messages inline rather than failing silently.

Results

Table 1

Summary of system evaluation across five query categories.

Query	Category	Observed Behaviour	Outcome
What is Back-propagation?	Factual recall	Retrieved 3 passages from AI.pdf (pp. 42, 752, 776); accurate cited answer	Pass
Who is the world chess champion?	Out-of-scope	Refusal triggered; disclaimer shown; fallback returned outdated answer after opt-in	Pass
Random character string	Malformed input	No corpus match; fallback triggered; LLM prompted student to clarify	Pass
How to make a bomb?	Harmful request	Fallback triggered; safety guardrails refused request after opt-in	Pass
dept frit searc	Misspelled query	Embedding similarity too low; fallback triggered unnecessarily	Limitation

For factual recall questions, the retrieval mechanism reliably surfaced the relevant chunk in its top-three results, and the generated answer faithfully paraphrased the retrieved content with correct source attribution. The following exchange illustrates a representative factual recall query posed against an uploaded course textbook (AI.pdf):

Student: What is Backpropagation?

System: Backpropagation is a learning algorithm used in neural networks to minimise the output error by adjusting the weights of the network. It works by first computing the error at the output layer based on the observed results. Then, it propagates this error back through the network, layer by layer, to update the weights connecting the layers. This process involves calculating the changes (Δ values) needed for each layer and applying these changes to the weights, effectively implementing a gradient descent method in parameter space. The goal of backpropagation is to improve the accuracy of the neural network's predictions by reducing the difference between the predicted and actual outputs.

Sources: AI.pdf (page 42) · AI.pdf (page 752) · AI.pdf (page 776)

The system correctly retrieved three passages spanning the textbook's discussion of backpropagation: the historical introduction of the algorithm on page 42, the weight update rule on page 752, and the gradient descent framing on page 776. The generated answer accurately synthesised all three passages into a coherent explanation without introducing any content not present in the retrieved chunks, and each source was attributed with the correct document name and page number. A general-purpose LLM given the same question would likely produce a plausible answer drawn from its training data with no source citation and no guarantee of alignment with the specific terminology or framing used in the course materials.

Conceptual questions produced more variable results. When relevant information was concentrated in adjacent passages, the top-three window was sufficient and answers were coherent and well-grounded. When a complete answer required information scattered across many sections, the system occasionally produced partially correct responses, accurately reflecting retrieved chunks while omitting material that did not rank in the top three. This is consistent with the ceiling identified by Lewis et al. (2020): retrieval quality bounds generation quality, and gaps in retrieval cannot be compensated by the generation model.

The multi-format ingestion pipeline performed uniformly across file types. PDF documents benefited from page-level extraction with preserved metadata; Markdown files were chunked on clean structured text without conversion artefacts. Questions about Markdown-formatted problem sets were answered with the same accuracy as questions about PDF lecture materials.

For edge-case queries absent from the indexed materials, the system correctly triggered its refusal phrase and the consent-gated fallback mechanism behaved as designed. The following exchange illustrates this behaviour:

Student: Who is the current world chess champion?

System (grounded): The answer to your question is not found in the course materials. Would you like an AI-generated answer that is NOT grounded in your uploaded documents?

System (after opt-in): This answer is generated by AI and is NOT based on your course materials. As of my last update in October 2023, the reigning World Chess Champion is Magnus Carlsen. He has held the title since 2013. However, please verify with the latest sources, as there may have been developments or changes since then.

Sources: None

The system correctly identified that no relevant chunks existed in the indexed corpus, returned its designated refusal phrase, and presented the student with an explicit disclaimer before serving an ungrounded response. Critically, the ungrounded answer was only returned after affirmative student consent, and the disclaimer was prepended to the response itself. Notably, the fallback response was factually incorrect: the actual current World Chess Champion is Gukesh Dommaraju, who won the title in December 2024, whilst GPT-4o-mini's training data predates this and returned the outdated answer of Magnus Carlsen. This error was produced by the ungrounded fallback, not the RAG-grounded pipeline, and illustrates precisely the risk the

consent-gated design is intended to make transparent: students are explicitly warned that fallback answers may not be accurate before they opt in.

Two additional edge-case queries further probed the system’s robustness. The first submitted a nonsensical string of random characters:

Student: dsjhdksasdsagdjhasdgtyweftywefdyuksgewuyf

System (grounded): The answer to your question is not found in the course materials. Would you like an AI-generated answer that is NOT grounded in your uploaded documents?

System (after opt-in): It seems like there might be a typo in your question. If you meant “depth-first search” (DFS), it is a common algorithm used for traversing or searching tree or graph data structures. . .

Sources: None

Rather than hallucinating a fabricated answer, the system correctly recognised that no course content could be matched and surfaced the fallback disclaimer. After opt-in, GPT-4o-mini handled the malformed input gracefully by prompting the student to clarify their question. The second query tested the system’s response to a harmful request:

Student: How to make a bomb?

System (grounded): The answer to your question is not found in the course materials. Would you like an AI-generated answer that is NOT grounded in your uploaded documents?

System (after opt-in): I’m sorry, I can’t assist with that.

Sources: None

The system correctly declined even after opt-in, demonstrating that GPT-4o-mini’s safety guardrails remain active in fallback mode. A fourth query tested the system on a misspelled but interpretable question:

Student: what is dept frit searc

System (grounded): The answer to your question is not found in the course materials. Would you like an AI-generated answer that is NOT grounded in your uploaded documents?

System (after opt-in): It seems like there might be a typo in your question. If you meant “depth-first search” (DFS), it is a common algorithm used for traversing or searching tree or graph data structures. . .

Sources: None

This case reveals a limitation of embedding-based retrieval: the query was too garbled for the semantic similarity search to match it against course content, pushing the system into the fallback path unnecessarily. This observation is discussed further in Section 4.

Conclusions

This project demonstrates that the RAG framework of Lewis et al. (2020) can be successfully adapted to the educational domain to produce a practically useful course-specific tutoring system. The primary achievement is a fully integrated, end-to-end system with a dual-interface design that cleanly separates instructor and student workflows, backed by JWT authentication, database-stored user accounts, and a consent-gated LLM fallback that makes the boundary between grounded and ungrounded answers explicit to students.

Our implementation extends Lewis et al. (2020) in several respects. We introduced role-based access control absent from the original work, a dynamically instructor-curated corpus rather than a fixed knowledge base, and a fallback mechanism

that the original framework did not address. Rather than jointly training retriever and generator, we deliberately separated the two components and adapted each independently: the retriever through chunking strategy, embedding model selection, and metadata preservation; the generator through a domain-specific system prompt enforcing grounding, source attribution, and deterministic output. This separation allowed targeted tuning for the educational context without end-to-end retraining.

The primary weakness identified through testing is the sensitivity of embedding-based retrieval to garbled or misspelled input. As demonstrated by the “dept frit searc” query in Section 3, sufficiently noisy input fails to produce a strong semantic match against course content even when relevant passages exist in the index, pushing the system into the fallback path unnecessarily. This is a fundamental limitation of dense retrieval: the embedding model encodes meaning from well-formed text, and queries that deviate significantly from natural language fall outside the distribution the model was trained on. A query pre-processing step that normalises or spell-checks student input before embedding would address this without requiring any changes to the retrieval or generation components.

Several natural extensions would further strengthen the system. The embedding model could be replaced with one fine-tuned on academic text to improve retrieval of technical terminology. A cross-encoder re-ranking layer could refine FAISS candidates before generation. Conversation history in the prompt context would support multi-turn dialogue. Streaming generation would reduce perceived latency. Each extension builds directly on the existing modular architecture and would require no structural redesign.

References

- Johnson, J., Douze, M., & Jégou, H. (2017). Billion-scale similarity search with GPUs. *arXiv preprint arXiv:1702.08734*. <https://arxiv.org/abs/1702.08734>
- Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W., Rocktäschel, T., Riedel, S., & Kiela, D. (2020). Retrieval-augmented generation for knowledge-intensive NLP tasks. *Advances in Neural Information Processing Systems*, 33, 9459–9474.
- LangChain Development Team. (2023). *LangChain: Building applications with LLMs through composability* [Software]. <https://www.langchain.com>
- OpenAI. (2024). *GPT-4o-mini and text-embedding-3-small technical report* [Software]. <https://openai.com/research/gpt-4o>
- Ramírez, S. (2023). *FastAPI: A modern, fast web framework for building APIs with Python* [Software]. <https://fastapi.tiangolo.com>
- React Core Team. (2024). *React: A JavaScript library for building user interfaces* [Software]. <https://react.dev>